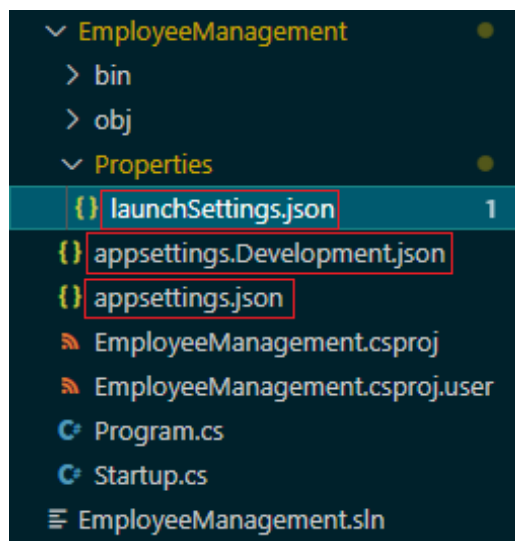


01.2 - Launchsettings and Appsettings

1. Launchsettings.json

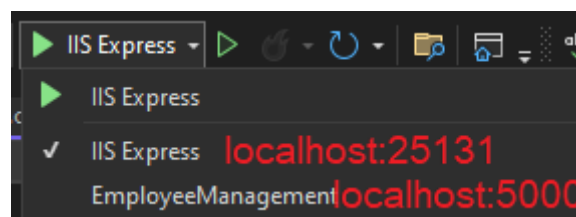
Launchsettings = Launch + Settings – настройките за стартиране

Solution Explorer -> Properties -> launchSettings.json – използва се само от локалната машина и не е необходим за публикуване, а за съхранение на определени настройки, независимо откъде се използва – VS/CLI/Code



- Application configuration;
- Environment specific app settings strategies and files
 - appSettings.Development.json - development environment
 - appSettings.Staging.json – staging environment
 - appSettings.Production.json – production environment
- Environment variables uses

За стартиране на проекта като localhost:номер с Ctrl+F5 или само F5 имаме 2 профила: IIS Express и вторият е с името на проекта, който сме избрали EmployeeManagement. Ако изберем първия, който е по подразбиране, минава през VS, а втория през CLI, който стартира нашата апликация. Нека го докажем. Затваряме проекта от CLI (за да не хвърля грешки) и избираме стартиране с IIS Express, тогава виждаме изпълнението на програмата с единия порт localhost:25131, а при избор за старт с името на нашата програма EmployeeManagement, излиза същият резултат през друг адрес localhost:5000.



Съответно този номер може да бъде променен от launchSettings.json.

```
{ } launchSettings.json 1 X
EmployeeManagement > Properties > { } launchSettings.json > { } profiles
1 {
2   "iisSettings": {
3     "windowsAuthentication": false,
4     "anonymousAuthentication": true,
5     "iisExpress": {
6       "applicationUrl": "http://localhost:25131",
7       "sslPort": 0
8     }
9   },
10  "profiles": {
11    "IIS Express": {
12      "commandName": "IISExpress",
13      "launchBrowser": true,
14      "environmentVariables": {
15        "ASPNETCORE_ENVIRONMENT": "Development"
16      }
17    },
18    "EmployeeManagement": {
19      "commandName": "Project",
20      "dotnetRunMessages": "true",
21      "launchBrowser": true,
22      "applicationUrl": "http://localhost:5000",
23      "environmentVariables": {
24        "ASPNETCORE_ENVIRONMENT": "Development"
25      }
26    }
27  }
28 }
```

ASPNETCORE_ENVIRONMENT със стойност Development може да бъде сменен със Staging/Production в зависимост от това в каква среда стартираме проекта – Staging/Production, също може да добавим нови променливи. Например в Startup.cs

```
EmployeeManagement > Startup (extension) IApplicationBuilder IApplicationBuilder.UseDeveloperExceptionPage() (+ 1 overload)
public class Startup
{
    public void Configure(IApplicationBuilder app)
    {
        app.UseDeveloperExceptionPage();
    }
}
```

Captures synchronous and asynchronous Exception instances from the pipeline and generates HTML error responses. This should only be enabled in the Development environment. !!!

Returns: A reference to the app after the operation has completed.

```

Startup.cs
EmployeeManagement > Startup.cs > Startup
13 public class Startup
22 public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
23 {
24     environment if (env.IsDevelopment()) .IsStaging .IsProduction
25     {
26         app.UseDeveloperExceptionPage();
27     }

```

стр. с много инфо за грешки - само при разработка

IsDevelopment() връща true ако "ASPNETCORE_ENVIRONMENT": "Development" и тогава се изпълнява UseDeveloperExceptionPage, т.е. ще се покаже тази HTML – генерирана страница с изключения, която съдържа извънредно много информация, затова не е подходяща за потребителите – те биха могли да злоупотребят с изписаните уязвимости.

```

if (env.IsDevelopment()) "ASPNETCORE_ENVIRONMENT": "Development"
{
    app.UseDeveloperExceptionPage();
}

```

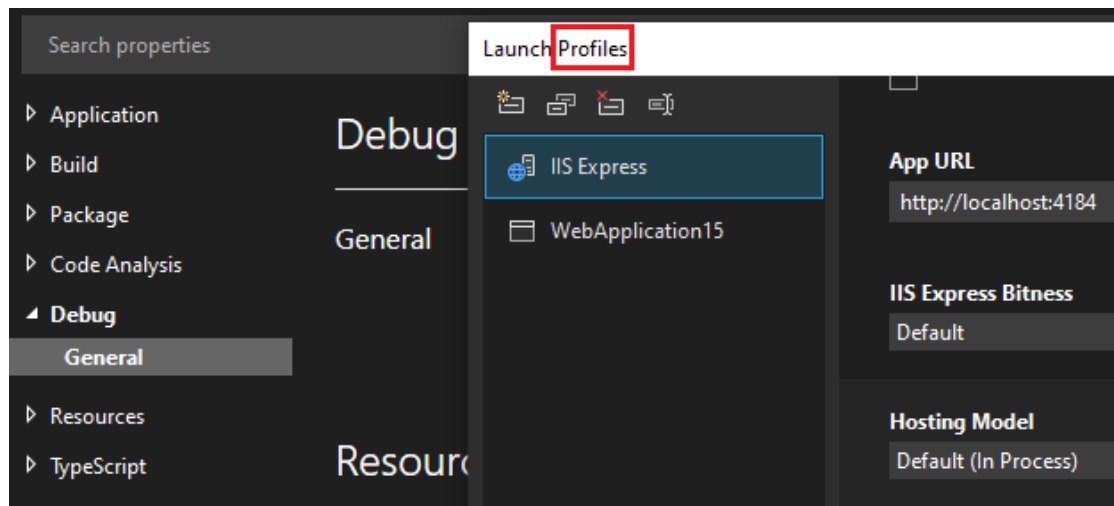
Environment може да заема 3 стойности:

- Development – по време на разработка като нас сега
- Staging – финалната стъпка преди публикуване
- Production – публикуван за потребителите

"commandName" може да приема стойности "IISExpress", "Project" и "IIS".
Project = EmployeeManagement

commandName	AspNetCoreHostingModel	Internal Web Server	External Web Server
Project	Hosting Setting Ignored	Only one web server - Kestrel	
IISExpress	InProcess	Only one web server - IIS Express	
IISExpress	OutOfProcess	Kestrel	IIS Express
IIS	InProcess	Only one web server - IIS	
IIS	OutOfProcess	Kestrel	IIS

User Interface: Д.Б. върху проекта и скрул до последно -> properties ->



2. [Appsettings.json](#)

Appsettings = Application + Settings – настройките на приложението

ASP.NET Core appsettings.json File

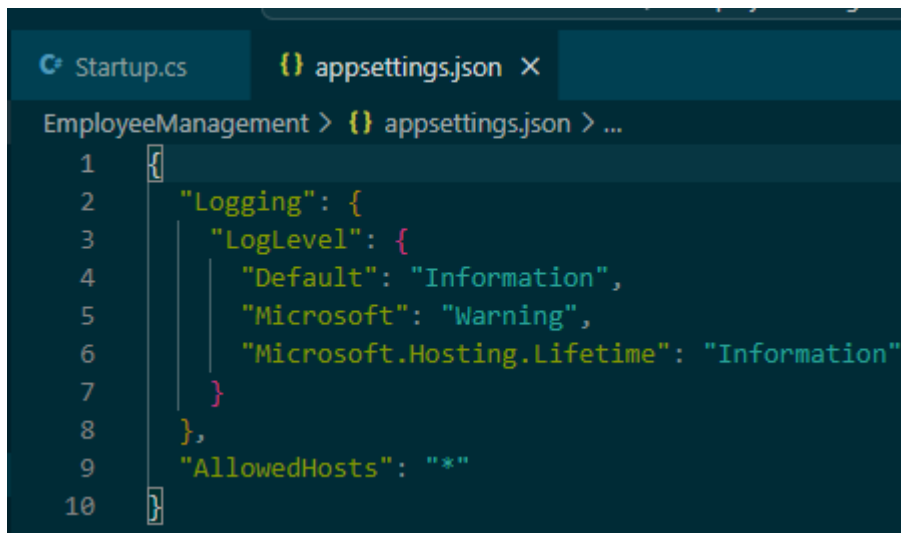
Configurations Sources in ASP.NET Core

- Files (appsettings.json, appsettings.{Environment}.json)
- User secrets
- Environment variables
- Command-line arguments

To access configuration information

- IConfiguration Service

Empty project template:



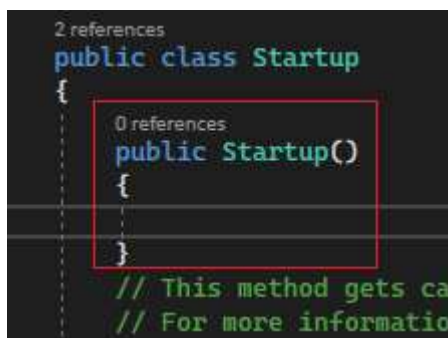
```
1  {
2    "Logging": {
3      "LogLevel": {
4        "Default": "Information",
5        "Microsoft": "Warning",
6        "Microsoft.Hosting.Lifetime": "Information"
7      }
8    },
9    "AllowedHosts": "*"
10 }
```

Конфигурационната информация в .net обикновено се съхранява като ключове.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*",
  "MyKey": "Value of MyKey from appsettings.json"
}
```

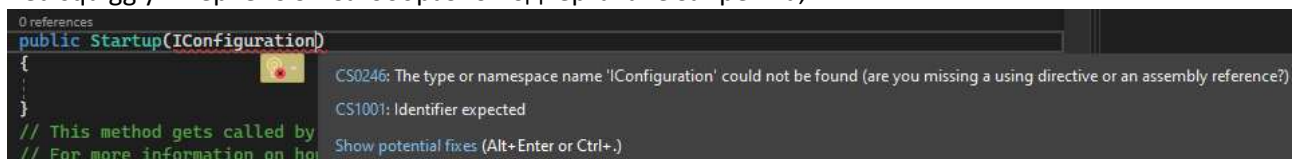
Конфигурационната информация в в appsettings.json обикновено се добавя с key value pairs.

И отиваме да го декларираме в Startup.cs, и за да бъде достъпен, инжектираме IConfiguration service, предоставен от framework в Startup class за работа с ключове. За това ни трябва constructor – ctor + Tab x 2 (2 последователни клавиша Tab) и съответно автоматично вгражда public Startup() {}



```
2 references
public class Startup
{
  0 references
  public Startup()
  {
  }
  // This method gets called by the runtime. Use this method to
  // configure the application.
}
```

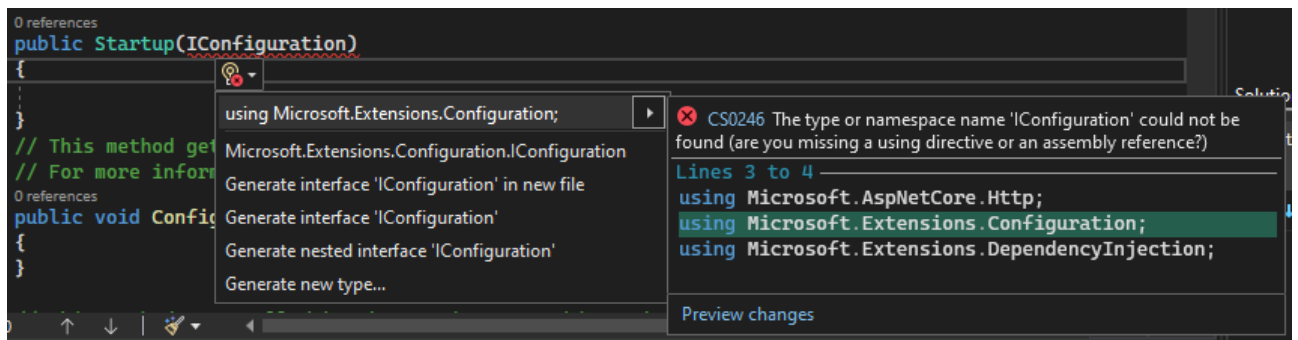
Red squiggly – червено зигзагообразно подчертаване за грешка,



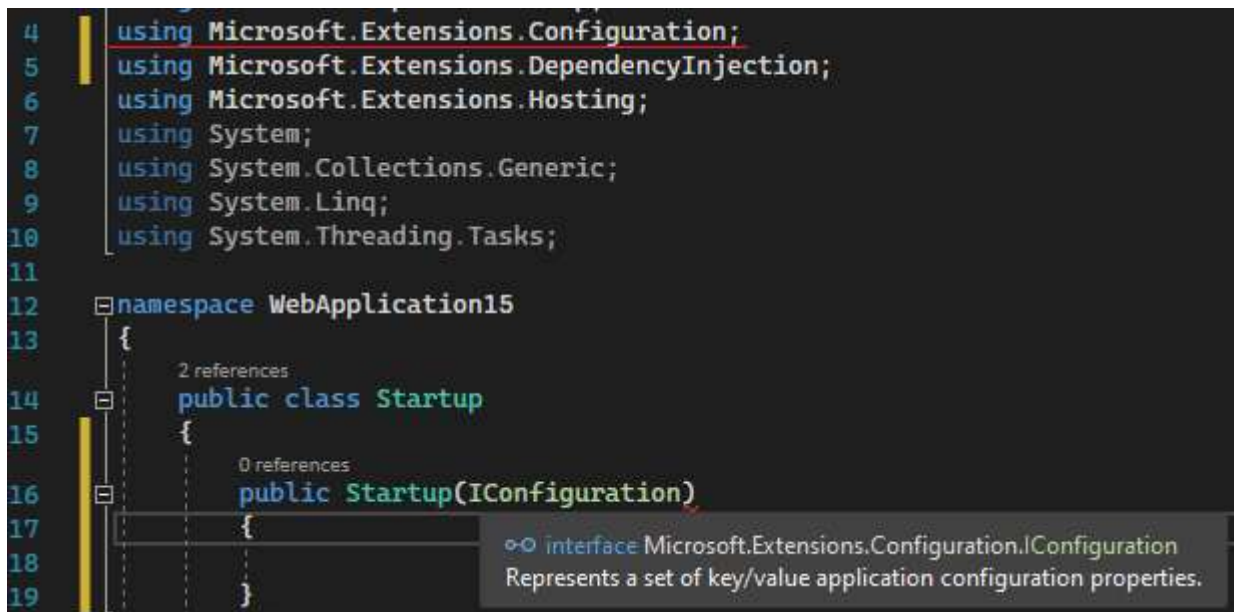
```
0 references
public Startup(IConfiguration)
{
}
// This method gets called by the runtime. Use this method to
// configure the application.

CS0246: The type or namespace name 'IConfiguration' could not be found (are you missing a using directive or an assembly reference?)
CS1001: Identifier expected
Show potential fixes (Alt+Enter or Ctrl+.)
```

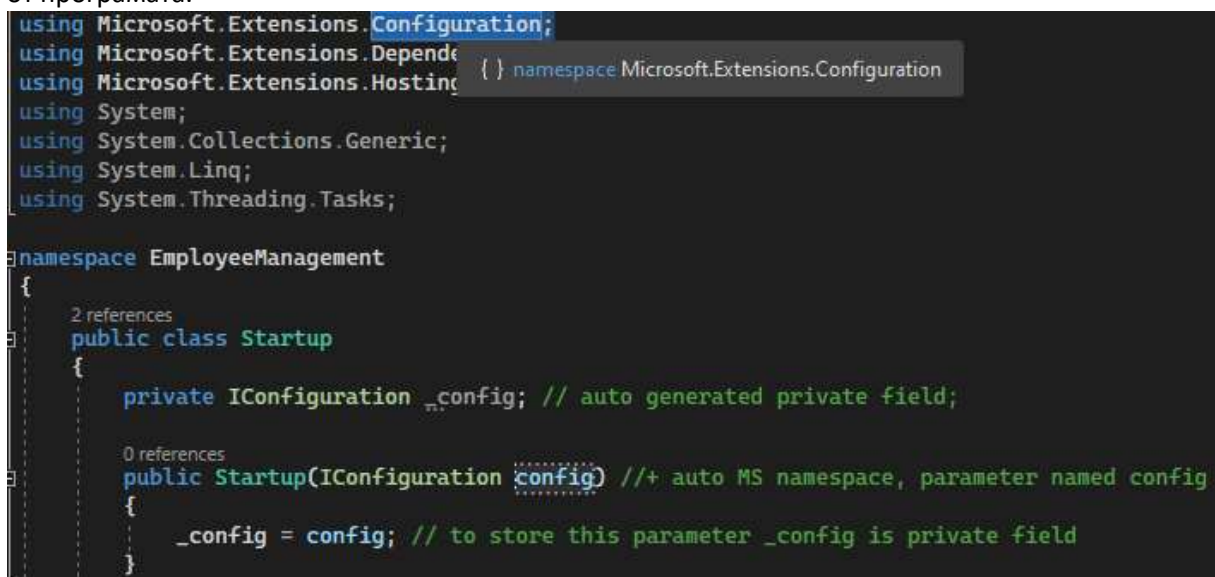
което с натиска на клавишна комбинация **Ctrl' + ,** (period) извежда съвети за корекциите.



Съответно при натискане на 'Enter' автоматично коригира с добавяне на липсващото namespace в началото



Въвеждаме променлива config и съответно автоматично генерираме private field поле от паметта за нея с Ctrl + . Обръщаме внимание, че използвайки зависимото инжектиране, за да вмъкнем IConfiguration service, а в предишните версии на .NET (под 2.2) е използван външен framework като Ninject StructureMap и др. В .net Core, той е цялостна (интегрална) част от програмата.



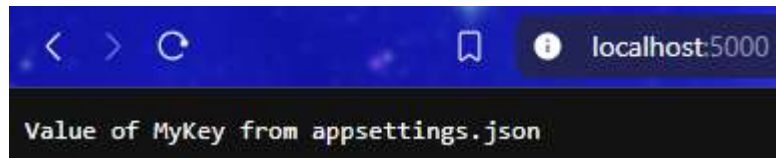

```
public Startup(IConfiguration config)
{
    _config = config;
}
```

Извикваме изписването на екрана:

```
await context.Response.WriteAsync(_config ["MyKey"]);
await context.Response.WriteAsync(_config ["MyKey"]);
```

Резултат: Value of MyKey from appsettings.json

Резултат:



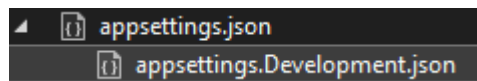
Готово – направихме конфигурационни настройки

* appsettings.json

По същият начин може да направим environment specific app settings file – appsettings.{Environment}.json, например:

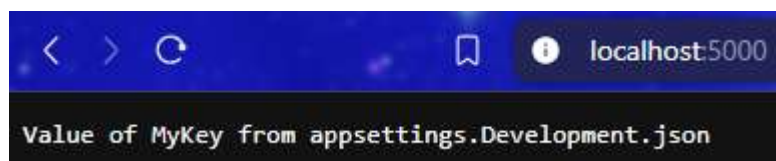
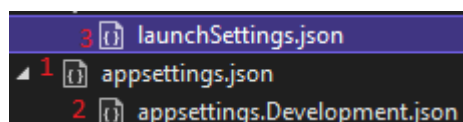
- + appsettings.Development.json,
- + appsettings.Production.json
- + appsettings.Stage.json

Важно е да запомним, че тези специфични настройки ще пренапишат основния файл appsettings.json



Т.е. свойствата, зададени в appsettings.Development.json са с по-висок приоритет от appsettings.json.

```
},
  "MyKey": "Value of MyKey from appsettings.Development.json"
}
```

Ако се включи и environment variable, тя става с най-висок приоритет и нея показва браузъра.

```
"ASPNETCORE_ENVIRONMENT": "Development",
"MyKey": "Value of MyKey from launchsettings.json as Environment variable"
},
}
```



```
10  "profiles": {
11    "IIS Express": {
12      "commandName": "IISExpress",
13      "launchBrowser": true,
14      "environmentVariables": {
15        "ASPNETCORE_ENVIRONMENT": "Development",
16        "MyKey": "Value of MyKey from launchsettings.json as Environment variable"
17      }
18    },
19    "EmployeeManagement": {
20      "commandName": "Project",
21      "dotnetRunMessages": "true",
22      "launchBrowser": true,
23      "applicationUrl": "http://localhost:5000",
24      "environmentVariables": {
25        "ASPNETCORE_ENVIRONMENT": "Development",
26        "MyKey": "Value of MyKey from launchsettings.json as Environment variable"
27      }
28    }
29  }
30 }
```

С максимален приоритет е CLI.

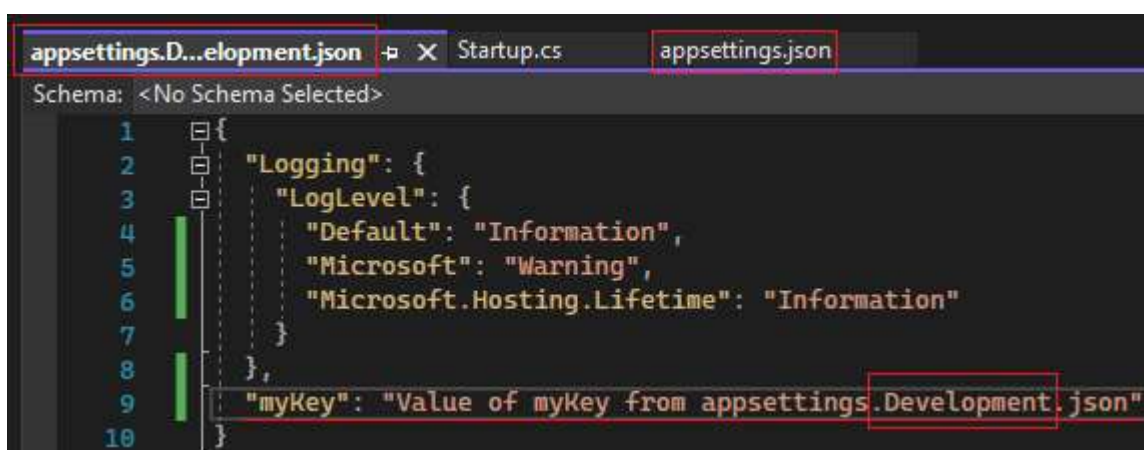
Към момента имаме едни и същи settings: MyKey на 3 места. Команда в CLI в папката на програмата:

```
dotnet run MyKey="The value from Commant Line"
```

Нека докажем, че настройките във специфичния appsettings.Development.json ще пренапишат основния appsettings.json. Копираме и задаваме да се покаже стойността в Development файла

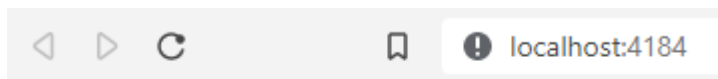
}, Слагаме запетайка преди ключа и необходимия ред код е преди последната фигурна скоба }

Вместо MyKey, може да се използвала myKey с малка буква – работи, но не е правилно.



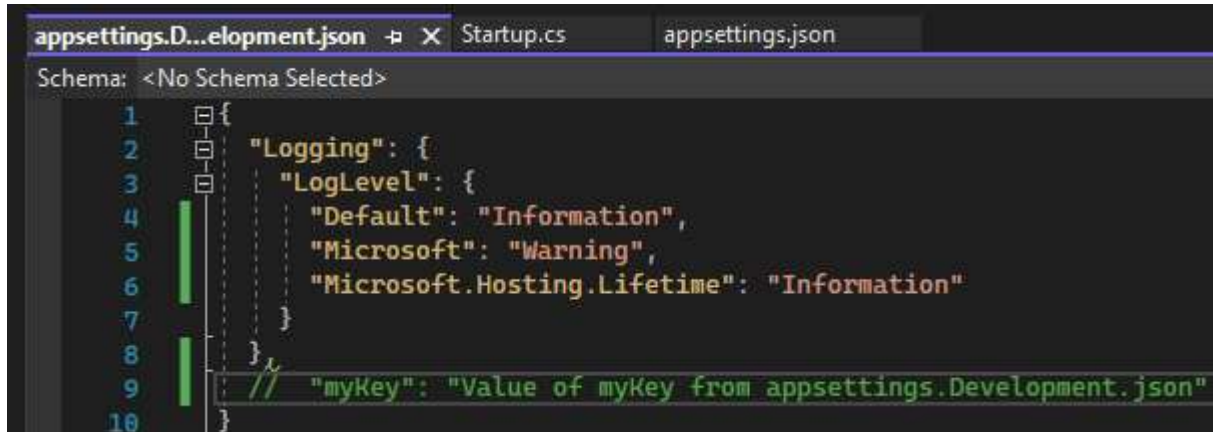
```
1  {
2    "Logging": {
3      "LogLevel": {
4        "Default": "Information",
5        "Microsoft": "Warning",
6        "Microsoft.Hosting.Lifetime": "Information"
7      }
8    },
9    "myKey": "Value of myKey from appsettings.Development.json"
10 }
```

Резултат: Development закрива основния .json:

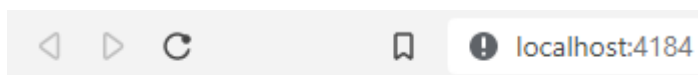


Value of myKey from appsettings.**Development**.json

Ако го изкоментираме (скрием за изпълнение)

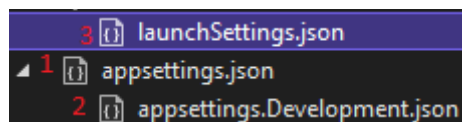


И рефрешнем браузера (дори без safe), appsettings.json става отново основен по подразбиране

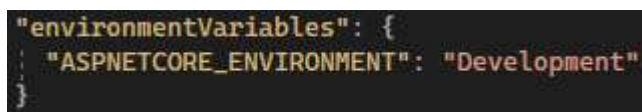


Value of myKey from appsettings.json

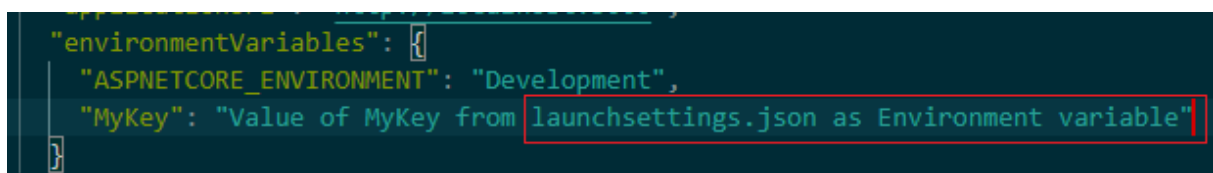
Като допълнение към файловете, информацията за конфигурациите идва и от потребителските user secrets, променливите на средата environment variables, които съхраняваме в локалния компютър в launchSettings.json



Обръщаме внимание, че в него вече имаме променливи на средата:



Сега ще включим друга променлива на средата със същия ключ като използвания вече myKey и сменим източника, от който да я вземем:



Важно е да запомним, че този IConfiguration service предоставен от framework, ще прочете конфигурационните файлове в този ред като последователност:

- 1) Files appsettings.json
 - appsettings.{Environment}.json

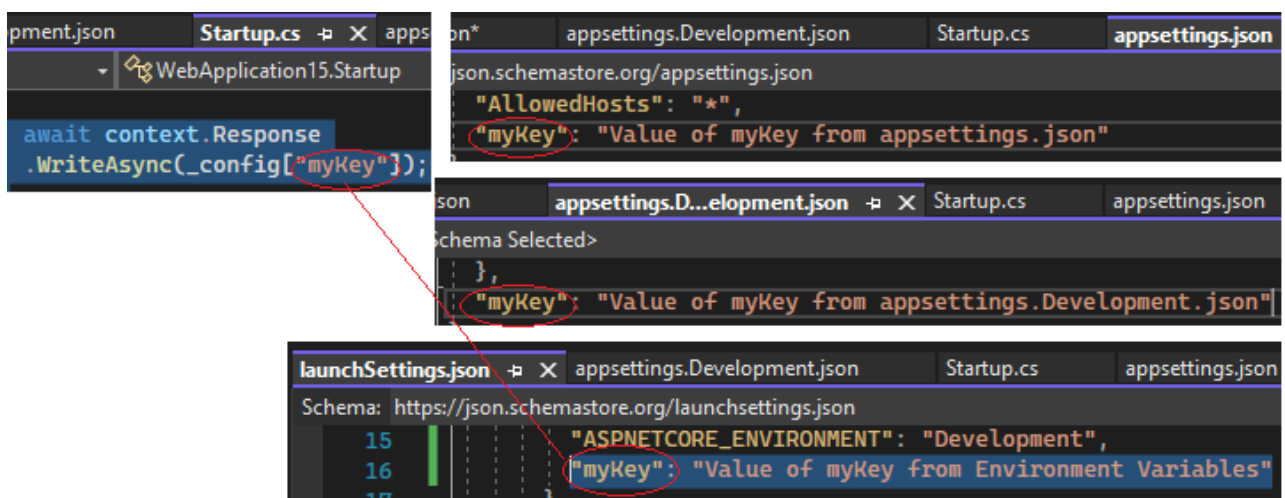
- 2) User secrets
- 3) Environment variables – launchsettings.json
- 4) Command-line arguments CLI (max)

Което означава, че Command-line arguments могат да презапишат environment variables, те от своя страна user secrets, те са с приоритет пред appsettings.{Environment}.json, а той засенчва генералния appsettings.json, т.е. по-късно създадените конфигурации се изпълняват с предимство пред вградените по подразбиране.

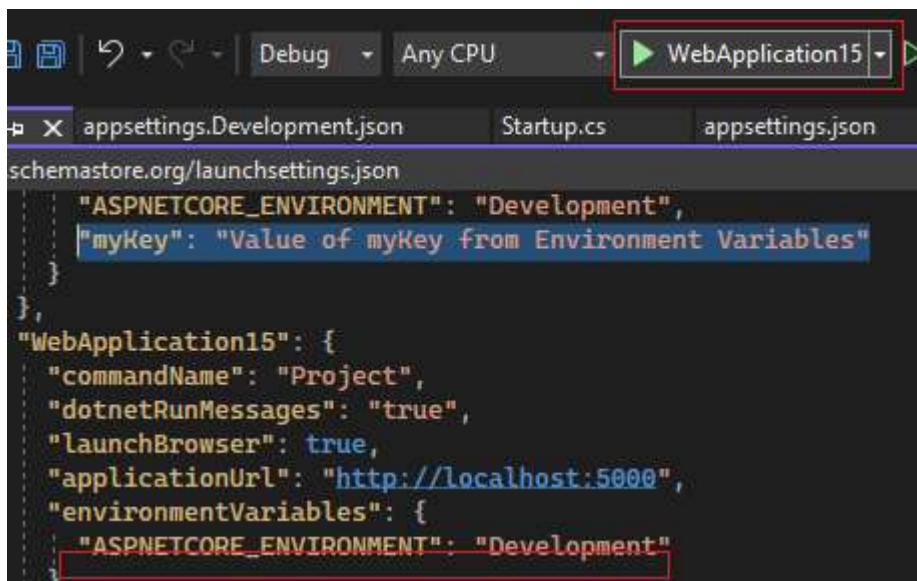
Нека го докажем:

Към момента разполагаме с 3 конфигурации от зададената последователност: appsettings.json, appsettings.{Environment}.json и environment variables. Това означава, че когато махнем коментара, Environment variable трябва да се изпълни вместо всички останали с IIS Express.

Тук съм тествала с малка буква за myKey, макар, че не е прието, работи.



Ако същата програма стартирам с WebApp15, понеже няма зададен ключ myKey в нея, ще се изпълни





Value of myKey from appsettings.Development.json

т.к. той е по подразбиране.

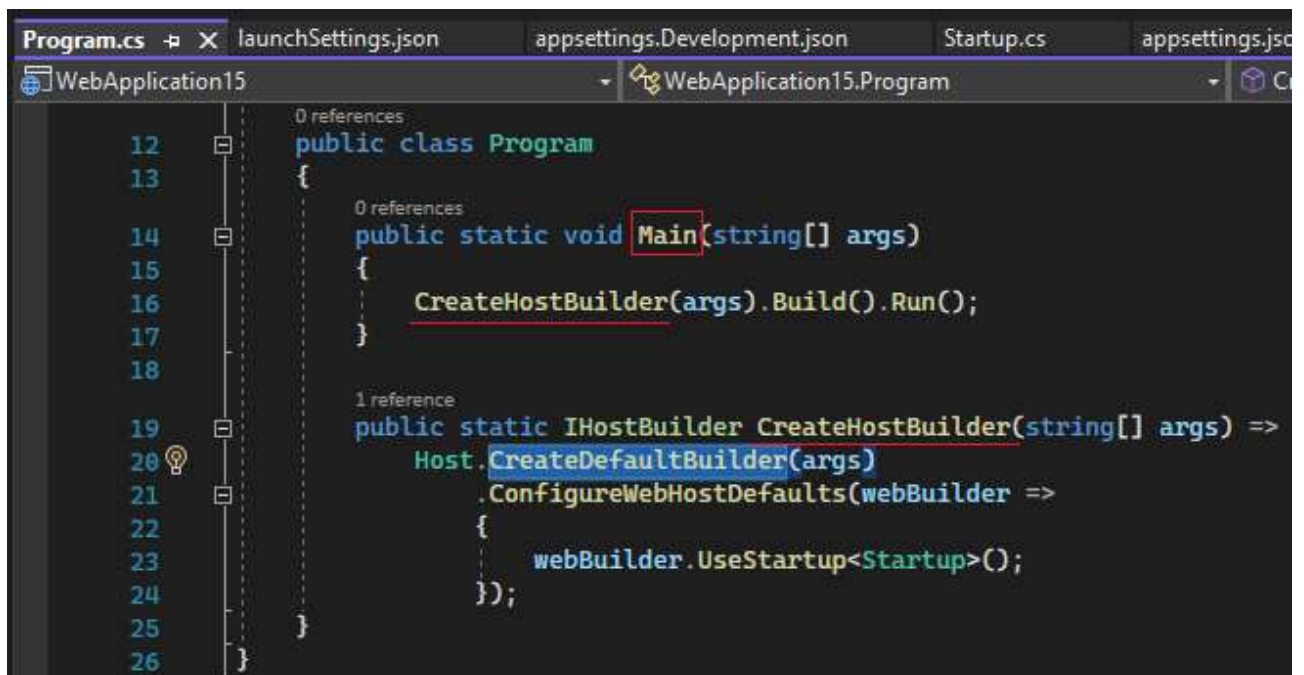
CLI: `dotnet run myKey="Value from Command Line"`

Зареждаме нов подпрозорец в браузъра с друг хостинг 5000, за резултатът от CLI:



С което доказваме, че командите в CLI са с най-висок приоритет от всички!

Program.cs файла съдържа Main-метод, който е входната точка на апликацията. Този метод извиква CreateWebHostBuilder, който от своя страна извиква CreateDefaultBuilder – това е методът, който съдържа наборът на разчитане на последователността по подразбиране на всички конфигурационни източници.



Понеже .NET core е open source, ние можем да намерим кода на този WebHostClass в github-библиотеката:

Версия 2.2 е максимална към момента:

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs>

Проверих за по-новата 5.0, но няма

The screenshot shows the GitHub repository for ASP.NET Core MetaPackages. The breadcrumb navigation indicates the path: `MetaPackages / src / Microsoft.AspNetCore / WebHost.cs`. A dropdown menu for switching branches is open, showing a list of branches: `release/1.0`, `release/1.1`, `release/2.0`, `release/2.1`, and `release/2.2` (which is selected and marked with a checkmark). The search bar in the dropdown contains the text "release". The background shows the code editor for `WebHost.cs`, with line 8 visible: `using Microsoft.AspNetCore.HostFiltering;`.

Приоритетът се определя от open-source метод `CreateDefaultBuilder` извикан във файл `Program.cs`

```
public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
        .ConfigureWebHostDefaults(webBuilder =>
        {
            webBuilder.UseStartup<Startup>();
        });
```

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore/WebHost.cs>

С търсачката в страницата търсим **ConfigureAppConfiguration** – методът, в който е описана последователността, която се залага по подразбиране за разчитане на всички

конфигурационни източници:

```
github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs
.ConfigureAppConfiguration((hostingContext, config) =>
{
    var env = hostingContext.HostingEnvironment;

    config.AddJsonFile("appsettings.json", optional: true, reloadOnChange: true)
        .AddJsonFile($"appsettings.{env.EnvironmentName}.json", optional: true, reloadOnChange: true);

    if (env.IsDevelopment())
    {
        var appAssembly = Assembly.Load(new AssemblyName(env.ApplicationName));
        if (appAssembly != null)
        {
            config.AddUserSecrets(appAssembly, optional: true);
        }
    }

    config.AddEnvironmentVariables();

    if (args != null)
    {
        config.AddCommandLine(args);
    }
}
```

Ако искаме, можем да променим тази последователност и дори да добавим наш собствен конфигурационен източник.